

Is Acceleration the New Accuracy? Real-Time Imitation Learning for Gesture Mimicry with a Compact Vision-Language-Action Policy on AMD Hardware

ABSTRACT

Vision-Language-Action (VLA) policies trained by imitation learning have made general-purpose robot control practical, but their real-world usefulness is gated less by peak accuracy than by *latency*: a policy that predicts the right motion too late produces the wrong motion. We study this coupling on a reactive *gesture-mimicry* task, in which an SO-101 robot arm running on LeRobot must imitate a human’s hand and wrist gestures observed through a single camera in real time. We fine-tune the compact 450M-parameter SmolVLA policy by behavior cloning on 2,000 teleoperated master-slave demonstrations collected from multiple people and backgrounds, and evaluate it by mean absolute error (MAE) between predicted and ground-truth joint trajectories. Because the policy emits chunks of 50 future actions from a single observation, we show—both analytically and empirically—that consuming more actions from a stale chunk before re-observing collapses accuracy (MAE grows from 3.3 at one action to 23.1 at forty), so the only way to stay accurate is to re-observe often, which in turn demands low inference latency. We then close the latency gap on AMD GPU (Strix/GPK-class) hardware through a sequence of systems optimizations: reducing flow-matching denoising steps from 30 to 2, compiling the full encoder-plus-prefill graph (not merely the denoiser) with `torch.compile`, casting to FP16 *before* compilation to avoid ROCm rescaling overhead, reducing the three-camera input to the single camera the task actually uses, and running Real-Time Chunking (RTC) asynchronous inference with a lowered refill threshold. Together these deliver roughly a $7\times$ throughput increase (2 to 13–15 inferences/s) and a $\sim 50\%$ MAE reduction (6 to ~ 3 , under the ~ 3.5 usability limit), turning an unusable policy into a smooth, deployable one. The result is a concrete demonstration that on AMD hardware, systems-level acceleration is a direct and measurable driver of task accuracy for reactive embodied AI.

Keywords: imitation learning, vision-language-action models, SmolVLA, action chunking, real-time inference, flow matching, ROCm, `torch.compile`, robot learning, edge AI.

1 Introduction

Recent Vision-Language-Action (VLA) models adapt pretrained vision-language backbones into robot policies that map camera images and a language instruction to low-level motor commands [3], [5], [9], [12]. Trained by imitation learning on teleoperated demonstrations, they generalize across objects and tasks far better than per-task policies. Yet a

property that is easy to overlook in benchmark tables becomes decisive on a physical robot: a VLA is a *closed-loop* system, and its usefulness depends not only on *what* it predicts but on *when* the prediction arrives. A policy that computes the correct action for the world it saw 300 ms ago is, on a moving target, simply wrong.

We make this coupling concrete on a deliberately unforgiving task: **gesture mimicry**. A human stands in front of a single camera and moves a hand up/down and left/right and opens or closes the wrist; an SO-101 arm running on the LeRobot stack [6] must *mimic* the gesture as it happens. Unlike pick-and-place, the target is a live, continuously moving human, so there is no “settling time”: every millisecond of policy latency is a millisecond of lag between operator and robot. This makes gesture mimicry an ideal probe for the thesis of this paper—that for reactive embodied AI, **acceleration is the new accuracy**.

The mechanism behind the thesis is action chunking. Modern VLAs [3], [12], [13] predict a *chunk* of n future actions from one observation to amortize inference cost and enforce temporal consistency. The more of that chunk the robot executes before capturing a fresh observation, the further it extrapolates open-loop from stale perception, and the worse it tracks a moving target. The only remedy is to re-observe and re-predict often—which is affordable only if each inference is fast. Latency and accuracy are therefore not independent axes; on this task they are the same axis.

This paper contributes: (1) a reactive gesture-mimicry benchmark and a 2,000-episode master-slave imitation-learning dataset for the SO-101 (Section 3); (2) an empirical characterization of the chunk-consumption effect, showing MAE degrading monotonically from 3.3 to 23.1 as more stale actions are executed, mirroring the SmolVLA paper’s own ablation and establishing the speed-accuracy identity (Section 5); (3) a description of the Real-Time Chunking asynchronous control loop that exploits low latency to keep the action queue fresh (Section 6); and (4) a sequence of AMD-specific systems optimizations—diffusion-step reduction, full-graph `torch.compile`, compile-aware FP16, single-camera reduction, and RTC tuning—that yield $\sim 7\times$ throughput and $\sim 50\%$ lower MAE on Strix/GPK-class hardware, taking the policy from unusable to deployable (Sections 7–8).

2 Background and Related Work

Imitation learning and action chunking. Behavior cloning learns a policy $\pi_\theta(a \mid o)$ directly from expert

demonstrations (o, a) , but small per-step errors compound over a trajectory [13]. Action Chunking with Transformers (ACT) mitigates this by predicting a short sequence of future actions as one unit, reducing the effective horizon over which errors accumulate and improving temporal smoothness [13]. Chunking is now standard across VLAs [3], [12], but it introduces the very staleness problem we study: a chunk predicted from o_t is executed over timesteps $t:t+n$, during which the world moves on.

Generative action heads. Rather than regressing a single action, recent policies model the action distribution with diffusion [7] or flow matching [10], which capture multi-modality and produce smooth trajectories. π_0 [3] and $\pi_{0.5}$ [8] attach a flow-matching “action expert” to a pretrained VLM; SmolVLA [12] follows the same recipe at a fraction of the size. The cost is an iterative denoising loop at inference time, which is a primary latency contributor and a main target of our optimization (Section 7).

Compact VLAs. Where OpenVLA [9] (7B) and π_0 [3] (3.3B) are large, SmolVLA [12] is a 450M-parameter open policy built on the efficient SmolVLM-2 backbone [11] and trained on community data, explicitly targeting single-GPU training and consumer-grade deployment. Its small size is what makes aggressive real-time optimization on a single AMD GPU tractable, so we adopt it as our base policy.

Real-time execution. Black et al. [4] formalize the latency problem and propose Real-Time Chunking (RTC): generate the next chunk while executing the current one, freezing the actions that will certainly run and “inpainting” the overlap for continuity. SmolVLA [12] ships a complementary asynchronous inference stack that decouples action execution from prediction. Our deployment builds on the LeRobot [6] implementation of these ideas and studies how far low latency, achieved through AMD systems optimization, extends their benefit.

3 The Gesture-Mimic Task and Dataset

3.1 Task Definition

A human operator performs hand gestures—raising and lowering the arm, moving left and right, and opening or closing the wrist—in front of a single RGB camera. The policy observes the camera frame and the arm’s current joint state and must drive an SO-101 6-DoF arm so that its end-effector pose mirrors the human’s gesture in real time (Figure 1). The task is purely reactive: there is no goal object and no fixed end state, so quality is measured by how faithfully and how promptly the arm tracks the live gesture.

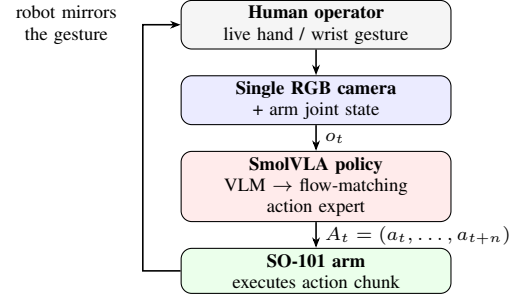


Figure 1. The gesture-mimic closed loop. A single camera observation o_t drives the SmolVLA policy, which emits a chunk of n future actions executed by the SO-101 arm. Because the operator moves continuously, any latency between observation and execution appears directly as tracking lag.

3.2 Data Collection by Master–Slave Teleoperation

We collect demonstrations by imitation learning in the classic teleoperation setup [13]. The arm ships as a *master–slave* pair connected by a cable: a human moves the master arm while watching the person in front of the camera, and the slave arm’s motors reproduce the master’s joint positions exactly. We record the camera video together with the synchronized slave-arm joint trajectory; time-aligning the two yields (image $\rightarrow$ action vector) pairs. Because the demonstrated action is the operator’s own gesture-tracking motion, the learned policy directly imitates human gesture-mimicking behavior—textbook behavior cloning.

The corpus comprises **2,000 episodes** spanning multiple demonstrators and varied visual backgrounds, aggregated from several LeRobot-format datasets. Validation uses a held-out set of 10 seen-speaker episodes. We report **mean absolute error (MAE)** between predicted and ground-truth joint trajectories (lower is better); qualitatively, motion becomes visibly unusable above an MAE of roughly **3.5**, which we treat as the usability threshold.

4 Policy Architecture

We fine-tune SmolVLA [12], whose two components are summarized here because they set up the latency budget we later attack.

Perception (VLM). A SmolVLM-2 backbone [11] (SigLIP vision encoder + SmolLM2 decoder) encodes the concatenation of image tokens, the language instruction, and a single state token obtained by linearly projecting the joint proprioception. Three efficiency choices make it small and fast: each frame is compressed to **64 visual tokens** via pixel shuffle; only the **first $N = L/2$ (16) decoder layers** are used to condition control; and the action expert interleaves **cross-attention** (to the VLM’s cached keys and values) with **causal self-attention** over action tokens.

Control (action expert). A ~ 100 M-parameter flow-matching Transformer [3], [10] of width $0.75\times$ the VLM hidden size predicts an action chunk $A_t = (a_t, \dots, a_{t+n})$ with $n = 50$. Inference integrates a learned velocity field over a small number of denoising steps (default 10). The

VLM prefix is encoded once and cached; the denoising loop reuses that cache. This structure means inference latency has two parts—a *one-time prefix/encoder cost* and a *per-step denoising cost*—both of which we reduce in Section 7.

5 Speed Is the New Accuracy

5.1 Why Consuming a Chunk Hurts Accuracy

A chunk A_t is predicted from a single observation o_t . If the robot executes k actions from A_t before capturing a new observation, then action a_{t+k} is applied to a world that has advanced k control periods past o_t . On a static task this open-loop extrapolation is cheap; on a live gesture it is corrosive, because the human has moved and the late actions track a pose that no longer exists. Accuracy therefore degrades monotonically with the number of *consumed* (stale) actions.

Table 1 shows exactly this on our validation set (SmolVLA v1, FP16, ~ 65 ms latency). MAE is 3.28 when a single fresh action is consumed per inference and climbs to 23.07 at forty—an $\sim 7\times$ degradation driven purely by staleness, not by any change in the model. The SmolVLA paper reports the same qualitative collapse in its execution-horizon ablation (success rate 80.3% $\rightarrow$ 51.8% as executed actions grow from 1 to 50) [12]; our continuous MAE metric makes the effect even sharper.

TABLE 1. MAE versus number of consumed (stale) action states, at fixed ~ 65 ms latency (SmolVLA v1, FP16). Executing more of a chunk before re-observing sharply worsens accuracy.

Consumed actions	MAE	Latency (ms)
1	3.28	65.16
2	3.43	65.37
4	4.29	67.72
5	4.56	66.81
10	4.80	65.00
20	6.21	64.05
30	12.09	62.83
40	23.07	64.33

5.2 The Latency–Reactivity Identity

The practical consequence is that accuracy demands *frequent re-observation*, and re-observation frequency is bounded by inference latency. Following the asynchronous-inference analysis [4], [12], to keep the action queue from emptying (and thus keep the robot from either stalling or running open-loop) the queue-refill threshold $g \in [0, 1]$ must satisfy

$$g \geq \frac{\mathbb{E}[\ell_S]}{n \Delta t}, \quad (1)$$

where ℓ_S is inference latency, Δt the control period, and n the chunk size. Equation (1) states the identity plainly: *lower latency ℓ_S permits a smaller g* , i.e. fewer inferences to stay reactive, or equivalently, at fixed g , a larger safety margin so the queue always holds fresh actions. Either way, cutting ℓ_S directly buys the ability to consume fewer stale actions per inference—which, by Table 1, is exactly what lowers MAE. This is why, on this task, **a faster model is a more accurate**

model. It is also why the fastest configuration in our sweeps is consistently the most accurate one, even at fixed model weights: lower latency lets the RTC loop overwrite stale actions with fresh predictions before they are ever executed (Section 6).

6 Real-Time Chunking and Asynchronous Inference

To turn low latency into fresh actions, we run the policy in an asynchronous Real-Time Chunking loop [4], [12] rather than the naive predict-then-execute-whole-chunk scheme. Execution and prediction are decoupled: a control thread pops actions from a thread-safe queue at a fixed rate and never blocks on the model, while an inference thread refills the queue in the background from the latest observation. A new chunk is requested *before* the queue drains (when its remaining fraction falls below g), and the new chunk is aggregated onto the unexecuted tail of the old one so motion stays continuous across chunk boundaries. Actions rendered stale by the time the new chunk lands are dropped by a latency-based delay of $\lceil \ell_S / \Delta t \rceil$ steps. Algorithm 1 sketches the loop.

Algorithm 1 Asynchronous Real-Time Chunking control loop

```

1: capture  $o_0$ ;  $A \leftarrow \pi(o_0)$  ▷ initial chunk
2:  $handle \leftarrow \text{null}$ 
3: for  $t = 1 \dots T$  do
4:    $a_t \leftarrow \text{POPFront}(A)$ ;  $\text{EXECUTE}(a_t)$ 
5:   if  $|A|/n < g$  and  $handle$  is idle then
6:     capture  $o_t$ 
7:     if  $\text{NEEDSPROCESSING}(o_t)$  then ▷ skip near-duplicates
8:        $handle \leftarrow \text{ASYNCFER}(o_t)$  ▷ non-blocking
9:     end if
10:  end if
11:  if  $handle$  completed then
12:     $\tilde{A} \leftarrow \text{result of } handle$ 
13:    drop first  $\lceil \ell_S / \Delta t \rceil$  actions of  $\tilde{A}$ 
14:     $A \leftarrow \text{AGGREGATE}(A, \tilde{A})$  ▷ overlap the tail
15:     $handle \leftarrow \text{null}$ 
16:  end if
17: end for

```

The knob that matters for accuracy is the refill threshold g (in our stack the `chunk_size_threshold`). Lowering it makes the loop request fresh chunks sooner, so each executed action is grounded in a more recent observation—but a low g is only *affordable* when ℓ_S is small, per Equation (1). Our AMD optimizations shrink ℓ_S ; the RTC loop then converts that headroom into reactivity, and reactivity into accuracy.

7 Systems Optimization on AMD Hardware

The base policy runs at ~ 2 inferences/s in eager FP32—far too slow for reactive mimicry. We reduce latency on a single AMD GPU (Strix/GPK-class, ROCm [1]) through the sequence in Table 2. The upstream release ships only bfloat16 plus `torch.compile` on the flow-matching denoiser [2], [12]; our work extends the optimization to the whole policy and to the ROCm-specific precision path.

TABLE 2. Optimization ladder on a single AMD GPU. Each row is cumulative.

Optimization	Effect	Throughput
Baseline (eager, FP32)	—	2.0 inf/s
Diffusion steps 30 $\rightarrow$ 2	fewer expert passes	4.5–5 inf/s
Full-graph <code>torch.compile</code>	fused kernels	8.5 inf/s
FP16 cast <i>before</i> compile	half compute	13–15 inf/s
3 $\rightarrow$ 1 camera input	less encode	speed + acc.
RTC + <code>chunk_threshold</code> ↓	fresher obs.	reactivity↑
Action smoothing (post)	de-jitter	smoothness↑

Diffusion steps 30 $\rightarrow$ 2. For this task the flow-matching field is well approximated in two integration steps; extra steps add latency without measurable accuracy gain. This alone more than doubles throughput.

Full-graph `torch.compile`. We rewrote the LeRobot SmolVLA modules so that the vision encoder and the LLM-prefix prefill compile into a single graph [2], not just the denoising step as upstream. This removes per-layer Python dispatch and lets the compiler fuse kernels across the whole forward pass, reaching ~ 8.5 inf/s.

Compile-aware FP16. Casting weights to FP16 *before* compilation is critical on ROCm: casting afterwards makes the compiler insert up/down-scale operations around every layer, and the resulting overhead eats the precision win. Casting first yields a clean half-precision graph and pushes throughput to 13–15 inf/s. The best deployable configuration is FP16 with max-autotune-no-cudagraphs full-graph compilation at ~ 65 ms standalone latency (Table 3); a one-time cold-start compilation cost is paid once and cached.

TABLE 3. Compile-mode comparison at 10 consumed actions. FP16 with full-graph compilation gives the lowest latency.

Precision	Compile mode	MAE	Latency (ms)
FP16	no-cudagraphs-full	4.80	65.00
FP16	reduce-overhead	4.60	77.20
FP16	eager	5.02	85.61
BF16	reduce-overhead	4.75	76.81
BF16	eager	5.03	90.53
BF16	no-cudagraphs-full	5.01	103.25

Single-camera reduction. SmolVLA accepts three camera streams, but the gesture task uses one; feeding the single active camera and dropping the two unused channels reduces encode cost *and* improves accuracy, since the model no longer conditions on blank inputs.

RTC and threshold tuning. With latency reduced, we enable the RTC loop (Section 6) and lower the refill threshold so a new prediction is triggered after only ~ 5 consumed actions, keeping executed actions fresh.

Action smoothing. A causal post-filter over the action stream (moving average or EMA over a short window) removes the residual jitter that a continuous action space with limited data can produce, giving smooth motion with no retraining.

8 Experiments and Results

8.1 Throughput and Accuracy

The optimization ladder (Table 2) raises throughput from 2 to 13–15 inf/s ($\sim 7\times$). Over the same development cycle, scaling data to 2,000 episodes and fine-tuning reduced MAE from ~ 6 to ~ 3 —below the ~ 3.5 usability limit and an $\sim 50\%$ improvement. Crucially, these two gains reinforce each other: the higher throughput is what allows the RTC loop to consume few stale actions (Table 1), so the deployed MAE reflects the fast, fresh-observation regime rather than the slow, stale one.

8.2 Model Version Progression

Table 4 tracks accuracy across releases. Reducing SmolVLA’s three-channel processing to a single channel (v1.1) and scaling from 1,000 to 2,000 raw episodes (v2) both improve MAE, with v2 giving the best mid-horizon accuracy (e.g. $3.94 \rightarrow 2.78$ at 10 actions). A later variant fine-tuned on YOLO gradient-background data (v3) did not improve behavioral smoothness, so v2 is our recommended release.

TABLE 4. MAE by version (30 FPS, ~ 31 ms, wrist-roll excluded). Single-channel processing (v1.1) and 2k-episode data (v2) each help.

Consumed actions	MAE v1.1	MAE v2
1	2.08	2.05
2	2.26	2.26
4	2.65	2.68
5	3.18	2.97
10	3.94	2.78
20	4.87	4.06
30	10.76	10.82
40	18.84	18.86

8.3 Behavioral Evaluation

Beyond MAE, we ran a scripted behavioral suite of eight gesture actions on the physical arm (Table 5). SmolVLA v2 passed all but one action and was rated “stable and smooth,” with gripper failures confined to one hand; v1.1 and v3 showed more frequent gripper failures and rougher motion. This confirms that the MAE improvements translate into qualitatively better on-robot behavior.

TABLE 5. On-robot behavioral pass/fail across eight gesture actions.

Action	v1.1	v2	v3
Rest	Fail	Pass	Fail
Right $\rightarrow$ side (close/open)	Pass	Pass	Pass
Right $\rightarrow$ top (close)	Pass	Pass	Pass
Left $\rightarrow$ side (close)	Pass	Pass	Pass
Left $\rightarrow$ side (open)	Pass	Pass	Pass
Right $\rightarrow$ home (close/open)	Pass	Pass	Pass
Left $\rightarrow$ top (close)	Fail	Fail	Fail
Left $\rightarrow$ home (close/open)	Pass	Pass	Pass
Smoothness	Moderate	Smooth	Poor

8.4 Control-Loop Throughput

Table 6 reports end-to-end rates. Inference throughput rises from ~ 16 to ~ 21.5 inf/s across versions in performant mode, while the control loop sustains ~ 24 – 27 FPS—comfortably real-time for gesture tracking on a single AMD GPU.

TABLE 6. Inference and control-loop rates (FPS).

Version	Eager		Performant	
	Infer.	Loop	Infer.	Loop
v1.1	15.95	27.33	17.98	24.40
v2	17.88	27.09	21.53	22.08
v3	18.54	26.77	20.42	23.52

9 Discussion and Limitations

Our results support a simple but consequential claim: for reactive embodied AI, systems-level acceleration is not a mere convenience but a direct driver of task accuracy, because it lets the closed loop stay grounded in fresh perception. The gesture-mimic task makes the effect legible, but the mechanism—stale action chunks degrade tracking, and only low latency permits frequent re-observation—applies to any policy that chunks actions against a changing world [4].

Several limitations remain. Validation uses seen-speaker episodes; an unseen-speaker split would better measure generalization. Our headline latency numbers are for SmolVLA; a controlled three-way comparison against ACT [13] and $\pi_{0.5}$ [8] on identical hardware would strengthen the systems argument. The aggregation function in our RTC loop is a practical extension of the published algorithm [4], [12] rather than a formally analyzed variant. Finally, the two-step flow-matching approximation is tuned to this task’s dynamics and may need revisiting for higher-precision manipulation.

10 Conclusion

We presented a real-time imitation-learning system for gesture mimicry built on the compact SmolVLA policy and deployed on a single AMD GPU. By collecting 2,000 master-slave teleoperation demonstrations and fine-tuning by behavior cloning, then attacking inference latency with diffusion-step reduction, full-graph `torch.compile`, compile-aware FP16, single-camera reduction, and Real-Time Chunking, we achieved $\sim 7\times$ throughput and $\sim 50\%$ lower MAE, moving the policy below the usability threshold and onto a physical arm with smooth, reactive behavior. The central finding is an identity rather than a trade-off: on reactive tasks, acceleration *is* accuracy, and AMD hardware plus ROCm systems optimization is a practical path to both.

References

- [1] AMD. ROCm: Open software platform for gpu compute. AMD ROCm Documentation, 2025. <https://rocm.docs.amd.com>.
- [2] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, et al. PyTorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pages 929–947, 2024.
- [3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. arXiv preprint arXiv:2410.24164, 2024. <https://arxiv.org/abs/2410.24164>.
- [4] K. Black, A. Z. Ren, M. Equi, and S. Levine. Real-time execution of action chunking flow policies. arXiv preprint arXiv:2506.07339, 2025. <https://arxiv.org/abs/2506.07339>.
- [5] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. arXiv preprint arXiv:2307.15818, 2023. <https://arxiv.org/abs/2307.15818>.
- [6] R. Cadene, S. Alibert, F. Capuano, M. Aractingi, A. Zouitine, P. Kooijmans, J. Choghari, M. Russi, C. Pascal, S. Palma, M. Shukor, J. Moss, A. Soare, D. Aubakirova, Q. Lhoest, Q. Gallouédec, and T. Wolf. LeRobot: An open-source library for end-to-end robot learning. In International Conference on Learning Representations (ICLR), 2026. <https://arxiv.org/abs/2602.22818>.
- [7] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In Robotics: Science and Systems (RSS), 2023. <https://arxiv.org/abs/2303.04137>.
- [8] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. arXiv preprint arXiv:2504.16054, 2025. <https://arxiv.org/abs/2504.16054>.
- [9] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. OpenVLA: An open-source vision-language-action model. In Conference on Robot Learning (CoRL), 2024. <https://arxiv.org/abs/2406.09246>.
- [10] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In International Conference on Learning Representations (ICLR), 2023. <https://arxiv.org/abs/2210.02747>.
- [11] A. Marafioti, O. Zohar, M. Farré, M. Noyan, E. Bakouch, P. Cuenca, C. Zakka, L. Ben Allal, A. Lozhkov, N. Tazi, et al. SmolVLM: Redefining small and efficient multimodal models. arXiv preprint arXiv:2504.05299, 2025. <https://arxiv.org/abs/2504.05299>.
- [12] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and

R. Cadene. SmolVLA: A vision-language-action model for affordable and efficient robotics. arXiv preprint arXiv:2506.01844, 2025. <https://arxiv.org/abs/2506.01844>.

- [13] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In Robotics: Science and Systems (RSS), 2023. <https://arxiv.org/abs/2304.13705>.

DRAFT